

Guía de trabajo

Producto de la Unidad 1 — Matriz de decisión tecnológica · Bases de Datos III

Cada sección trae un recordatorio corto y un ejemplo, si algo no les quedó claro en la explicación en vivo, esto les debería alcanzar para seguir solos.

Recordatorio rápido: familias y motores

MongoDB — documental: documentos tipo JSON sin esquema fijo. Sirve cuando los datos varían mucho entre registros (catálogos).

Redis — clave-valor: todo en memoria RAM, una clave que apunta a un valor. Sirve cuando la velocidad es crítica (cachés, sesiones).

Neo4j — grafos: nodos y relaciones guardadas como conexiones directas. Sirve para redes y recomendaciones.

Cassandra — columnar: datos distribuidos en muchos servidores. Sirve para volúmenes de escritura enormes (sensores, logs).

PostgreSQL — relacional (SQL): tablas fijas con filas y columnas. Sirve cuando la consistencia es innegociable (pagos, transacciones).

Nombres del grupo: Jackeline Jaramillo Salazar

Fecha: 30 de septiembre de 2026

1. Caso real

Plantilla: "[Sistema] que maneja [tipo de datos] para [quién lo usa]". Tiene que ser específico, no sirve "una empresa" o "una app".

Ejemplo

"Plataforma de streaming de música que maneja catálogo de canciones, historial de reproducción y recomendaciones, para usuarios finales."

Si no tienen un caso propio, tomen uno del banco de 8 casos de respaldo.

Plataforma para una alcaldía o curaduría que maneja reportes ciudadanos de daños en edificios y vías y el seguimiento de cuadrillas de reparación, para ciudadanos, funcionarios y cuadrillas de reparación.

2. Tabla comparativa de motores candidatos

Elijan UNA parte de su sistema (no todo a la vez). Para esa parte, elijan 2-3 motores candidatos y califiquen cada criterio como Alta/Media/Baja, con una justificación de una línea.

¿Qué pregunta cada criterio?

Consistencia — ¿qué tan grave es que el dato esté desactualizado por un instante? (grave, como en dinero → SQL; no grave, como un contador de likes → NoSQL puede.)

Latencia — ¿qué tan rápido necesito la respuesta: milisegundos, o puede tardar un poco más?

Escalabilidad — ¿cuánto va a crecer esto: cientos, miles, millones de registros?

Costo operativo — ¿cuánto cuesta mantener y operar ese motor a ese volumen?

Criterio	MongoDB (documental)	Redis (clave-valor)	Neo4j (grafos)
Consistencia	Media — consistente dentro de cada documento; eventual entre réplicas, aceptable porque un reporte no es dinero.	Media — muy consistente en un solo nodo, pero los datos viven en memoria y pueden perderse si falla sin persistencia.	Alta — transacciones ACID; más de lo que este caso exige.
Latencia	Baja (rápida) — lecturas por índice en milisegundos al filtrar por severidad o zona.	Baja (la más rápida) — menos de un milisegundo, pero sobrada para consultar reportes.	Media — muy buena al recorrer relaciones; para listados simples no aporta ventaja.
Escalabilidad	Alta — crece con sharding si hay miles de reportes tras un evento como un sismo.	Media — limitada por la RAM disponible; consultar por campos no es su fuerte.	Media — crece bien en vertical; repartir un grafo en varios servidores es más complejo.
Costo operativo	Medio — Atlas tiene plan gratuito para empezar; las fotos se guardan aparte y aquí solo su enlace.	Alto — guardar todo el histórico en RAM sale más caro que en disco.	Medio — requiere aprender Cypher y más recursos para un caso que casi no usa relaciones.

Ejemplo de referencia (catálogo de canciones de una app de streaming) — la suya no tiene que verse igual, es solo para que vean cómo se llena:

Criterio	MongoDB (documental)	PostgreSQL (SQL)	Cassandra (columnar)
Consistencia	Baja — eventual, aceptable en un catálogo	Alta — más de lo que este caso exige	Media — eventual, ajustable
Latencia	Media — buena para lecturas frecuentes	Media-alta — JOINS con metadatos variables cuestan	Baja — pensada para escritura masiva
Costo operativo	Medio	Medio — paga funciones que aquí no se usan	Alto — más nodos para tolerancia a fallos

3. Justificación final

Plantilla: "Para _____, elegí _____ porque _____ es la prioridad, evidenciado por _____." Tiene que citar el criterio Y una evidencia concreta de su caso — no basta con decir "uso MongoDB".

Ejemplo

"Para el catálogo de canciones, elegí MongoDB porque la variedad de metadatos es alta y cambia por género — un esquema fijo obligaría a columnas vacías o tablas separadas por tipo de contenido, evidenciado en que un track de jazz y uno electrónico casi no comparten atributos."

Motor 1 — Para los reportes ciudadanos de daños, elegí MongoDB porque la flexibilidad del esquema es la prioridad, evidenciado en que el reporte de un edificio (número de pisos, material, grietas en columnas) y el de una vía (tipo de pavimento, longitud del hueco, tránsito afectado) casi no comparten campos, y además cada reporte lleva un número distinto de fotos.

Motor 2 — Para los reportes ciudadanos de daños, descarté Redis porque el costo operativo y la durabilidad pesan más que la latencia, evidenciado en que los reportes deben conservarse por años como evidencia, y guardar todo ese histórico en memoria RAM sería caro y una falla sin persistencia podría perder reportes. Lo uso en cambio para el estado de las cuadrillas, donde la latencia sí es la prioridad.

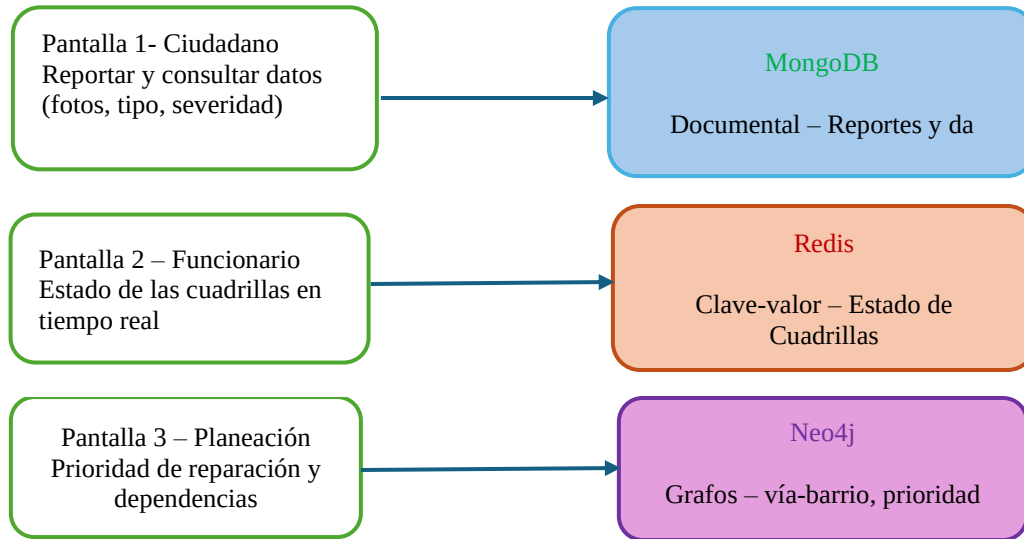
Motor 3 — Para los reportes ciudadanos de daños, descarté Neo4j porque la consulta principal es filtrar y listar reportes por severidad, tipo y zona y no recorrer relaciones, evidenciado en que un reporte individual es útil sin estar conectado a otros. Lo reservo para las dependencias entre infraestructuras (qué vía conecta a qué barrio), donde las relaciones sí son la prioridad.

4. Arquitectura inicial

Pregunta guía: si tu aplicación tuviera 3 pantallas principales, ¿qué pantalla llama a qué base de datos? No necesitan UML formal, un diagrama de bloques simple basta.

Ejemplo

Pantalla de búsqueda → MongoDB (catálogo). Pantalla de reproductor → Redis (qué está sonando). Pantalla de recomendaciones → Neo4j (grafo de gustos). Tres cajas, tres flechas.



5. Repositorio Git

¿Por qué ahora, aunque no haya código? Ese primer commit ya cuenta como evidencia de trabajo desde el día uno — en la Unidad 8 van a necesitar el historial completo.

- ☐ Repositorio creado en GitHub
- ☐ git clone — traje el repositorio a mi computador
- ☐ git add . — marqué los archivos para guardar
- ☒ git commit -m "..." — guardé los cambios con un mensaje
- ☐ git push — subí los cambios a GitHub

URL del repositorio: <https://github.com/jajaramillo09-debug/Matriz-Desicion-Tecnol-gica.git>